

课程笔记

CPU、SIMD 和 GPU

2026 南京大学《操作系统原理》第 20 讲

lecture-to-notes

2026 年 5 月 18 日



视频作者/频道：绿导师原谅你了（蒋炎岩 / jyy）

发布日期：2026-05-12

视频时长：1:29:45

视频链接：<https://www.bilibili.com/video/BV1df536aEMk/>

目录

1 本讲定位：推翻”CPU 顺序执行” 这个统治性假象	2
1.1 回顾：什么是 CPU?	2
1.1.1 ILP 实测：变量依赖如何影响 IPC	3
1.2 本章小结	3
2 SIMD：一条指令打多个数据	4
2.1 SIMD = Single Instruction, Multiple Data	4
2.1.1 演化轨迹	4
2.2 你”无意中”已经写过 SIMD	5
2.3 本章小结	5
3 GPU：从游戏机到通用计算	6
3.1 游戏机的”图形危机”	6
3.2 Shader：从固定数据结构到编程语言	6
3.3 SIMT：让一个 PC 管多个线程	7
3.3.1 SIMT vs SIMD：一字之差，世界不同	7
3.4 CUDA Programming Model	8
3.5 本章小结	9
4 总结与延伸	10
4.1 讲者核心观点	10
4.2 自测三问	10
4.3 与课程其他部分的 synthesis	10
4.4 拓展阅读	10

1 本讲定位：推翻”CPU 顺序执行”这个统治性假象

前面所有讲座 (13-19) 都把 CPU 当成一个无情的、顺序执行指令的机器：rv32ima_step()——取指令、译码、执行、写回，一条接一条。本讲要把这个模型彻底颠覆：

- 现代 CPU 早已在**内部并行**（指令级并行 / ILP）；
- CPU 还配备了 **SIMD** 指令集（一条指令操作多个数据）；
- 真正把并行做到极致的是 **GPU**——一条指令同时驱动几千个线程。

贯穿本讲的一句话

顺序执行只是 CPU 一个**精心维护的假象**。

当我们决心揭开这个假象，并允许硬件**显式地**并行——就有了 SIMD 和 GPU 这两条改变计算机系统命运的演化路径。

1.1 回顾：什么是 CPU？

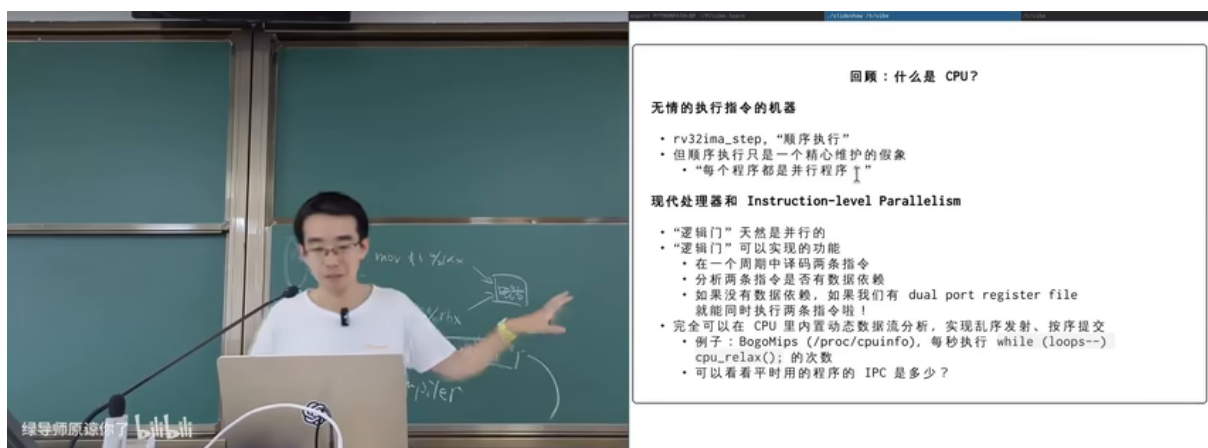


图 1: 回顾：CPU = ”无情的执行指令的机器” rv32ima_step()——顺序执行。但顺序执行是一个**精心维护的假象**：“每个程序都是并程序！”现代处理器通过逻辑门天然的并行性实现 **Instruction-Level Parallelism (ILP)**：一个周期内译码多条指令、分析依赖、若无依赖则同时发射；可以乱序发射 / 按序提交¹

逻辑门为什么”天生并行”

你们在数字逻辑课上用 Logisim 连电路时，所有门是同时计算的——信号从输入”光速”传播到输出。这意味着 CPU 内部完全可以放很多很多个加法器、比较器、寄存器读端口——从内存一次取出 32 / 64 / 128 个 bit、同时译码 4 / 8 条指令都没问题。**硬件并行是物理实现的本能。**

¹视频画面时间区间：00:07:15–00:07:30。

1.1.1 ILP 实测：变量依赖如何影响 IPC

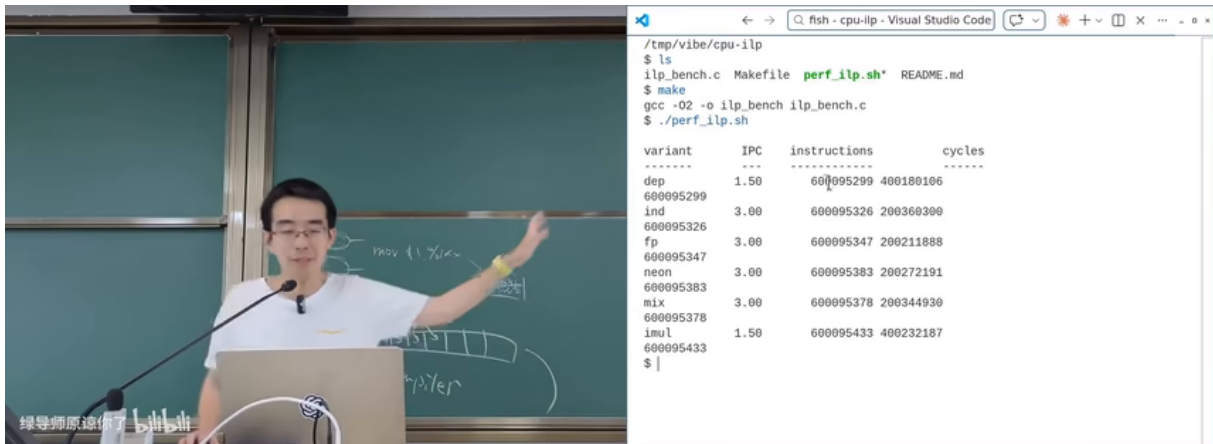


图 2: ILP 实测脚本输出：同样 6 亿条指令，不同 variant 的 **IPC**（Instructions Per Cycle）相差悬殊——dep（强依赖链）IPC=1.50；ind（独立指令）IPC=3.00；fp（浮点）3.00；imul（整数乘法）1.50。现代 CPU 在一个周期里发射 3 条独立指令是常态²

结论：你写的“X = 1; Y = 2;”在 CPU 里大概率就是同时执行的——只要这两条指令没有数据依赖。这就是 **Instruction-Level Parallelism**。

1.2 本章小结

CPU 早已不再“顺序执行”——硬件天然并行，编译器和 CPU 的乱序引擎一起把 ILP 利用到极致。但 ILP 是**隐式**的并行，受限于指令窗口大小。要做得更猛，我们需要**显式**的并行——这就是下面 SIMD 和 GPU 的故事。

²视频画面时间区间：00:12:15–00:12:30。

2 SIMD：一条指令打多个数据

2.1 SIMD = Single Instruction, Multiple Data

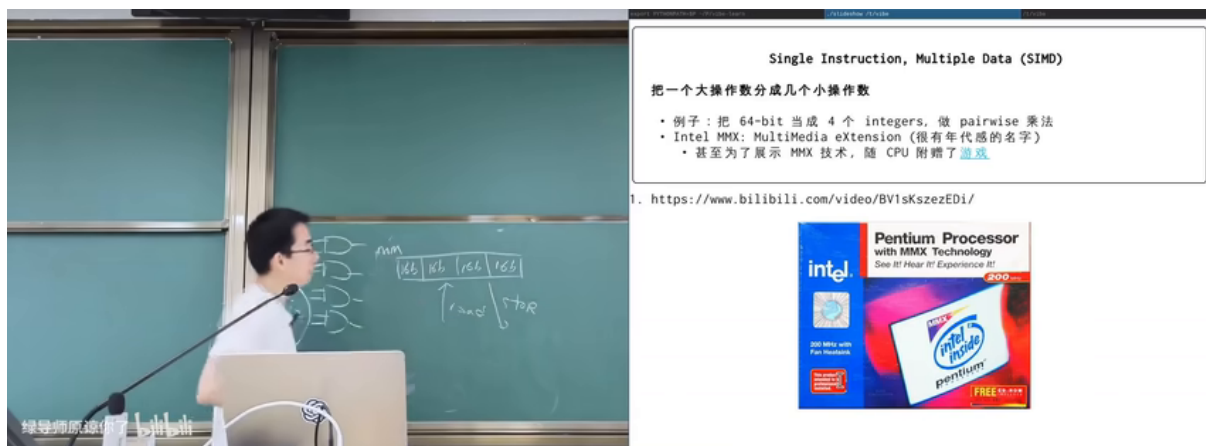


图 3: SIMD 概念：把一个大操作数分成几个小操作数同时计算。例：把 64-bit 整数当成 4 个 16-bit 整数做 pairwise 乘法。**Intel MMX (MultiMedia eXtension, 1997)**——非常有年代感的名字。随 Pentium MMX 处理器发售的盒子里还附赠了一张展示 MMX 技术的游戏光盘³

SIMD 的设计要点

- 新加一组宽寄存器（MMX 给 32 位机加 8 个 64-bit 的 MM0-MM7）；
- 新加一组打包指令（paddw, pmullw, ...）——能同时对寄存器里“几格 16-bit”做加法 / 乘法；
- 加 / 乘溢出特殊处理——不会进位到相邻“格”，而是 saturate 到最大值（像浮点饱和和那样）。

SIMD 的“killer app”就是**多媒体**：图像处理、视频解码、3D 渲染——每个像素的运算独立、又恰好可以打包成 4 / 8 / 16 个一组。

2.1.1 演化轨迹

- **MMX (1997)**：64-bit，复用浮点寄存器；
- **SSE (1999)**：128-bit XMM，独立寄存器；
- **AVX (2011)**：256-bit YMM；
- **AVX-512 (2016)**：512-bit ZMM；
- **ARM NEON / Apple Silicon** 也有等价物。

³视频画面时间区间：00:26:15–00:26:30。

2.2 你”无意中”已经写过 SIMD

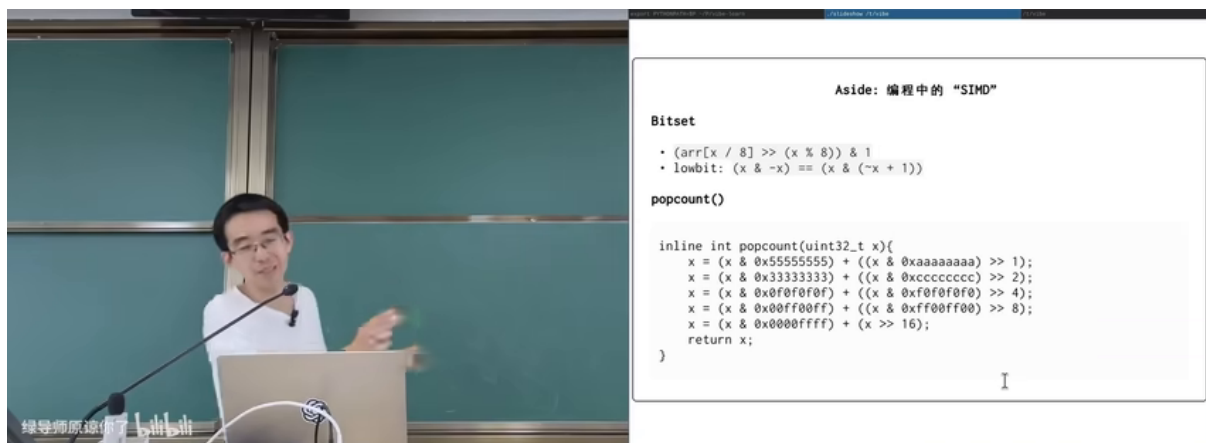


图 4: Aside ——你早已”亲手写过 SIMD”: bitset 的 $(arr[x/8] \gg (x\%8)) \& 1$ 就把一个 `uint8_t` 当成 8 个 bool 用; popcount 经典算法用一个 32-bit 字模拟 32 个并行加法器——那段 $x = (x \& 0x55555555) + ((x \& 0xaaaaaaaa) \gg 1)$ 就是这种思想⁴

Subword Parallelism

这种”在一个机器字里同时操作多个子字段”的技巧,叫 *subword parallelism*。早在硬件 SIMD 出现前,程序员就用 bitwise 运算手工模拟它——bitset、popcount、Bloom filter、bit-packed integer 等都是这一脉。SIMD 指令集只是把这种”打包 + 同步操作”固化到硬件里,让编译器或人手写汇编可以直接用。

2.3 本章小结

SIMD 把”对独立数据做相同操作”的场景从软件循环(每条标量指令一个数据)压缩成”一条向量指令 N 个数据”——吞吐量瞬间提升 4-16 倍。但 SIMD 还是 CPU 模型——所有 lane 共享一个 PC、共享内存、共享 cache。要把并行做到”几千个 lane”,需要完全不同的硬件——GPU。

⁴视频画面时间区间: 00:25:30–00:25:45。

3 GPU：从游戏机到通用计算

3.1 游戏机的”图形危机”

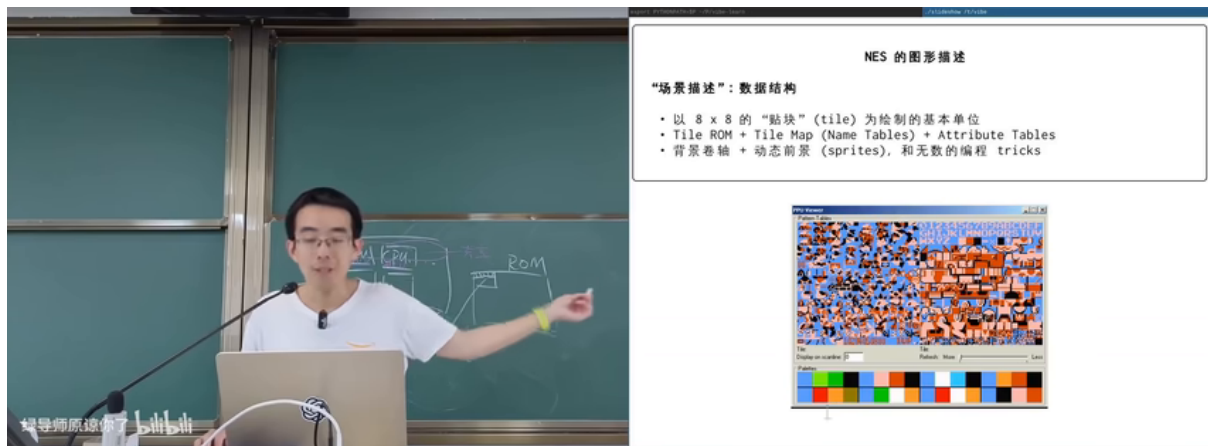


图 5: NES (任天堂红白机, 1983) 的图形描述: 用 8×8 的”贴块”(tile) 为绘制基本单位, 组合 **Tile ROM + Tile Map (Name Tables) + Attribute Tables**; 背景卷轴 + 动态前景 (sprites) 和无数的编程 tricks。《塞尔达传说》(1986) 这种”看起来精美”的游戏背后藏着巨大的”计算危机”——当画面越来越复杂, 固定渲染流水线再也不够用⁵

时代背景: 1972 Magnavox Odyssey (第一台家用游戏机, 只能显示光点) → 1983 NES (贴图 + sprite) → 1986 Zelda (平滑动画) → 1990s 3D 游戏爆炸 (DOOM, Quake) → 显卡厂商 (3dfx, NVIDIA, ATI) 军备竞赛。

3.2 Shader：从固定数据结构到可编程语言

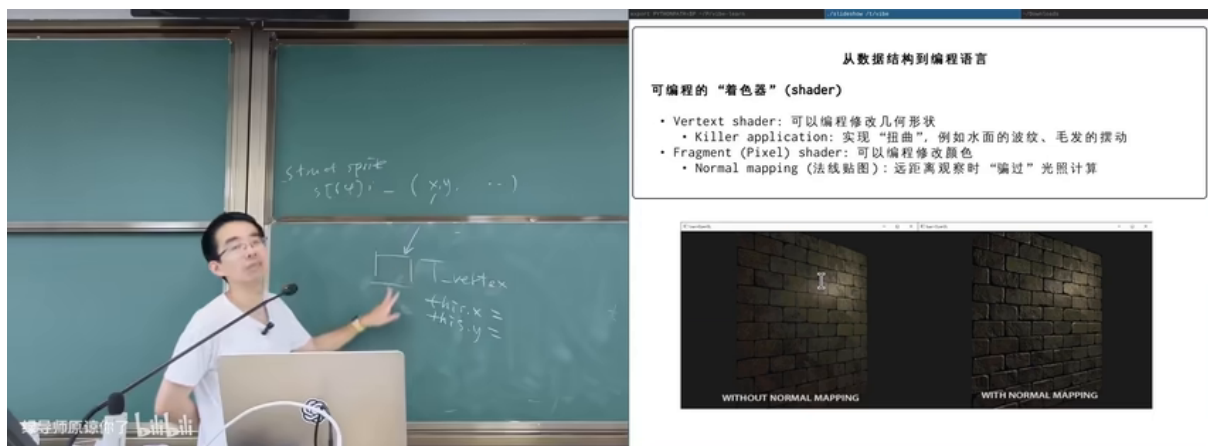


图 6: Shader (着色器) = 把”图形描述”从固定数据结构升级为可编程语言。**Vertex shader**: 可编程修改几何形状 (Killer app: 水面波纹、毛发摆动); **Fragment (Pixel) shader**: 可编程修改颜色 (Normal mapping 让平面墙看起来有立体凹凸——图中右侧砖墙就是 normal mapping 之后的效果)⁶

⁵ 视频画面时间区间: 00:46:21–00:46:36。

⁶ 视频画面时间区间: 01:04:00–01:04:15。

Shader 是 GPU 通用化的转折点

2001 年 NVIDIA GeForce 3 引入可编程 shader 之后，GPU 不再是“只能跑固定渲染管线”的专用芯片——它变成了一个大规模并行的计算平台。游戏开发者用 shader 写自定义视觉效果；几年后科研人员意识到：**shader = 通用并行计算函数**——只要把数据塞进纹理、读写 framebuffer，就能跑科学计算。这就是 GPGPU (General-Purpose computing on GPU) 的开端，直接催生了 2007 年的 CUDA。

3.3 SIMT：让一个 PC 管多个线程

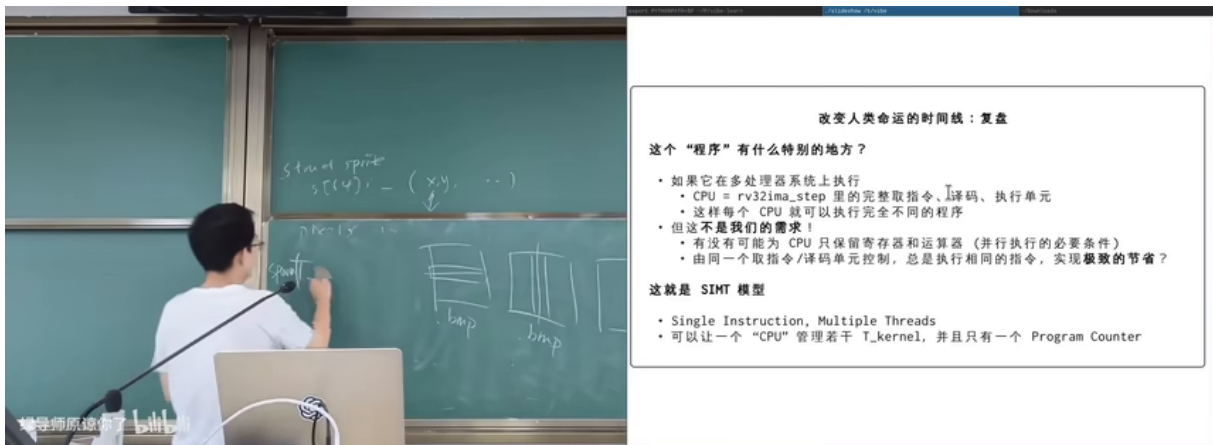


图 7：改变人类命运的时间线：SIMT = Single Instruction, Multiple Threads 模型。”如果 CPU 在多处理器系统上执行，每个 CPU 都执行完全不同的程序——但这不是我们的需求！有没有可能让 CPU 只保留寄存器和运算单元，由同一个取指/译码单元控制，所有 CPU 都执行相同的指令？这就实现了极致的节省。这就是 SIMT。”

一个 CPU 管理多个 T_kernel 函数实例，只有一个 Program Counter⁷

3.3.1 SIMT vs SIMD：一字之差，世界不同

- SIMD：一条指令操作多个数据，但它们在多线程内的不同 lane 里——程序员要显式打包 / 拆包；
- SIMT：一条指令同时驱动多个线程，每个线程像普通顺序程序一样写，硬件帮你把“同时执行同一指令的 N 个线程”绑成一个 warp。

⁷视频画面时间区间：01:08:43-01:08:58。

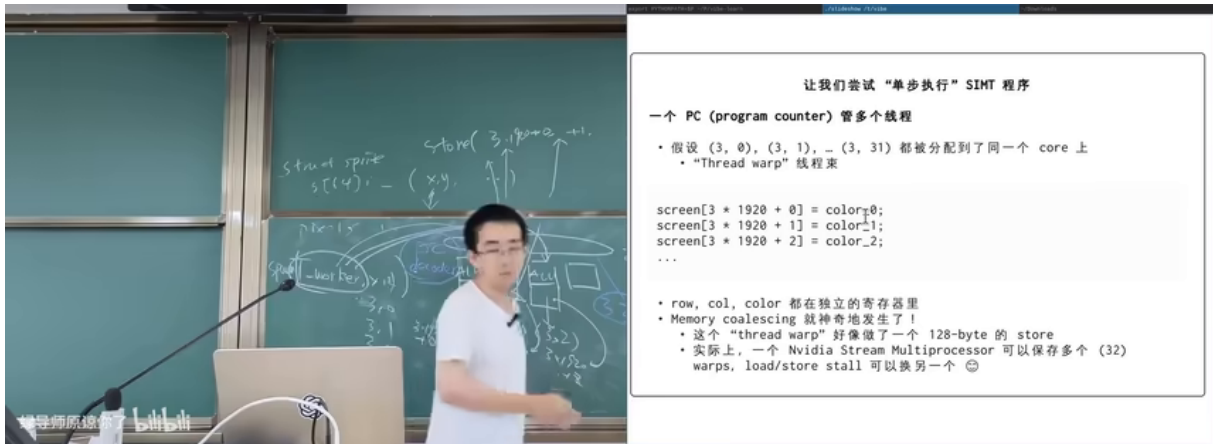


图 8: SIMT 程序的”单步执行”: 假设 (3, 0), (3, 1), ..., (3, 31) 都被分配到同一个 core 上, 组成一个 **Thread warp** 线程束。代码 `screen[3 * 1920 + 0/1/2] = color_0/1/2` 在 32 个线程里并行执行; row、col、color 都在**独立的寄存器**里; **Memory coalescing** 让 32 个相邻地址的 store 合并成一个 128-byte store。一个 Nvidia Stream Multiprocessor 可以同时保存多个 (32) warps——load/store stall 时换一个执行⁸

为什么 SIMT 这么省？

传统多核 CPU: 每个核**独立**的取指 + 译码 + 调度逻辑——这部分占 die area 的大头。32 个独立 CPU = 32 套这些逻辑。SIMT 的精髓: 32 个线程的取指 / 译码**共享一份硬件**, 每个线程只需要”自己的寄存器 + ALU”——die area 缩小 **4-8 倍**, 能在同样的芯片面积里塞 **8 倍** 多的算力。所以 NVIDIA H100 有 **14592 个 CUDA 核**——而最贵的 CPU 也就 96 核。

3.4 CUDA Programming Model

CUDA 把 SIMT 抽象成 C 的扩展:

```

1  __global__ void vec_add(float *a, float *b, float *c, int n) {
2      int i = blockIdx.x * blockDim.x + threadIdx.x;
3      if (i < n) c[i] = a[i] + b[i];
4  }
5
6  int main() {
7      // 启动 1024 * 1024 个线程, 每个算一个元素
8      vec_add<<<1024, 1024>>>>(d_a, d_b, d_c, n);
9  }

```

Listing 1: CUDA Kernel 示例: 向量加法

- <<<grid, block>>> 语法是 CUDA 独有的——**nvcc 把它编译成驱动调用**, 让 GPU 启动一组 thread blocks;
- 每个线程通过 `blockIdx` / `threadIdx` 知道自己是谁;
- 你写的**看起来是一个标量程序**——但硬件让一个 warp (32 个线程) **同时**走相同的指令路径。

⁸视频画面时间区间: 01:20:00–01:20:15。

Warp divergence

当 warp 里的线程走不同分支 (**if-else**)，硬件必须**分别**执行两条路径，掩盖不需要的线程。这叫 **warp divergence**，是 GPU 编程的性能杀手。写 CUDA 时要尽量让 warp 内所有线程走同样路径——”把 *if-else* 写成数学公式”是 CUDA 老手的经验。

3.5 本章小结

GPU 从”专用图形处理器”走到”通用大规模并行计算平台”，关键转折点是 **shader**（让渲染管线可编程）和 **SIMT**（让一个 PC 控制几千个线程）。今天 AI 训练 / 推理的核心算力**完全建立**在这两个抽象上——CUDA kernel 启动一组 thread blocks，每个 block 内 warp 同步执行 matmul。

4 总结与延伸

4.1 讲者核心观点

1. CPU 顺序执行是假象——逻辑门天然并行，现代处理器 $IPC \geq 3$ 是常态。
2. SIMD 是”硬件支持的 subword parallelism”——MMX $\rightarrow$ SSE $\rightarrow$ AVX $\rightarrow$ AVX-512 $\rightarrow$ ARM NEON 一路演化，多媒体是 killer app。
3. 你早就在写 SIMD——bitset、popcount、Bloom filter 都是手工 subword parallelism。
4. Shader 是 GPU 通用化的拐点——把”渲染数据结构”升级为”可编程函数”。
5. SIMT 比 SIMD 更省更猛——共享 fetch/decode, 独立 reg/ALU, 一片 H100 14592 个 CUDA 核 vs 最强 CPU 96 核。
6. CUDA 让你用 C 的语法写 SIMT——但 warp divergence 仍然是性能陷阱，需要程序员配合。

4.2 自测三问

看完本讲，能答上这三题吗？

1. 写一段 C 代码测量你的 CPU 上独立指令的 IPC (提示：长循环 + rdtsc / perf)。为什么浮点循环的 IPC 可以超过 1？
2. 用 SSE intrinsics (`_mm_add_epi32` 等) 改写一段 `for (i = 0; i < n; i++) c[i] = a[i] + b[i]` 的循环。编译后看汇编，验证它确实用了 SIMD。
3. 用 CUDA 写一个 1024×1024 矩阵相加 kernel；解释 `blockDim` / `gridDim` 怎么影响 warp divergence 和 memory coalescing。

4.3 与课程其他部分的 synthesis

- 与第 13 讲 (入门到放弃)：13 讲教你”为什么 `sum++` 算不对”——那是隐式的硬件并行带来的反直觉。本讲让显式并行成为主角。
- 与第 18 讲 (并行算法)：18 讲讲计算图怎么切；本讲给出切分后节点上的硬件支撑——SIMD 适合密集向量运算、SIMT 适合大规模独立任务。
- 与第 19 讲 (协程)：协程是用户态调度的轻量线程；warp 是硬件级调度的极轻线程——本质都是”把一个 PC 复用到多个执行流”。
- 与即将到来的第 21 讲：Token 的旅程会带你从 prompt 到 GPU kernel 全栈——本讲让你看清 CUDA matmul 在硬件层面真正发生了什么。

4.4 拓展阅读

- Hennessy & Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed ——第 4 章 *Data-Level Parallelism*。

- NVIDIA, *CUDA C++ Programming Guide* (官方文档, 是 GPU 编程入门最权威的材料) .
- Intel *Intrinsics Guide*: <https://www.intel.com/content/www/us/en/docs/intrinsics-guide/> ——SSE / AVX 所有指令的搜索引擎。
- Lindholm *et al.*, *NVIDIA Tesla: A Unified Graphics and Computing Architecture*, IEEE Micro 2008 (首次系统介绍 SIMT) .
- Akenine-Möller *et al.*, *Real-Time Rendering*, 4th ed ——把 vertex / fragment shader 历史讲得最清楚的一本.
- Henry S. Warren, *Hacker's Delight*, 2nd ed ——bitwise SIMD (popcount 等) 的圣经.